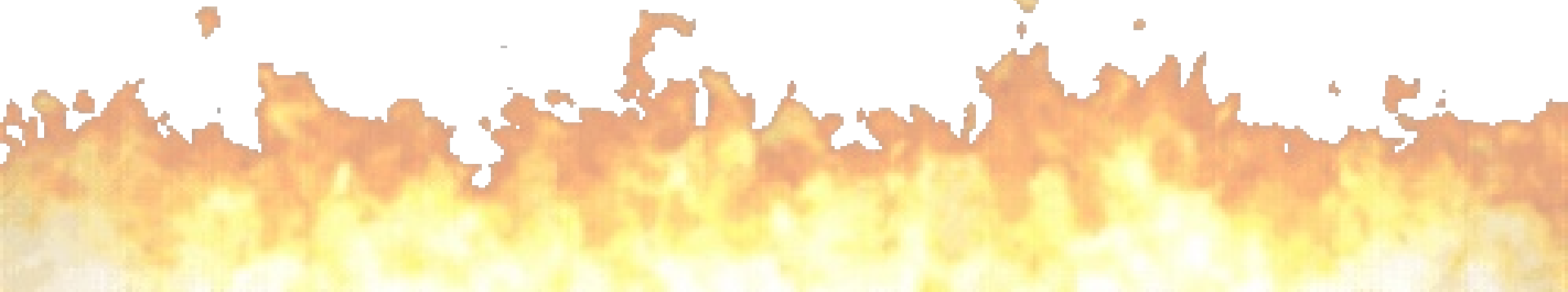


뉴런데브클래스

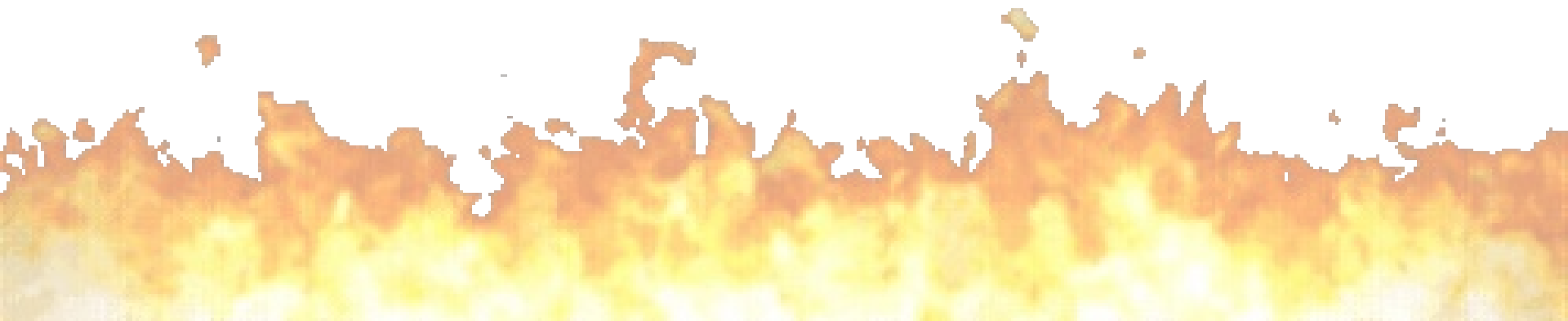
현대 AI의 원리와 구조

#4-2 벡터디비 실습



목차

- 1 실습 환경
- 2 인덱스 원리 - numpy 로 직접
- 3 pgvector 인덱스 측정
- 4 검색 품질 - 퓨전 · 분포 진단
- 5 정리



실습 개요

1. 강의의 인덱스 · 퓨전 · 분포 이론을 직접 만들고 숫자로 확인
 - 1 부 numpy 로 NSW · HNSW · IVF · PQ 를 처음부터 구현
 - 2 부 pgvector 로 같은 현상을 벡터 10 만 개에서 측정
 - 3 부 KLUE-MRC 로 RRF · Min-Max · 분포 진단
2. 모든 실습은 Jupyter 노트북
 - 노트북 12 개 , 실행하면 그래프 40 여 개가 그려진다
 - 슬라이드의 주장을 그래프로 검증 — 맞는 것과 다른 것을 구분
3. 4-1 과 같은 도커 컴포즈 하나
 - db · pgAdmin · Jupyter 가 함께 뜬다
 - 4-1 의 KLUE 데이터 · BM25 구현을 그대로 재사용
4. 두 가지 '정확도' 를 구분
 - 2 부 ANN recall@10: 정확한 k-NN 대비 몇 개를 찾았나 (인덱스의 근사 품질)
 - 3 부 Hit@10 · MRR: 정답 지문을 찾았나 (검색 자체의 품질)

시간	단계	노트북	계산 시간 (12 코어 실측)
00-10	① 환경	compose up → Jupyter → 00	Jupyter 이미지 빌드 1~2 분
10-40	② 1 부 원리 (numpy)	10 NSW · 11 HNSW · 12 IVF · 13 PQ	7 초 · 43 초 · 12 초 · 37 초
40-70	③ 2 부 pgvector	20 · 21 · 22 · 23 · 24	8 초 · 70 초 · 41 초 · 22 초 · 29 초
70-90	④ 3 부 품질	30 퓨전 · 31 분포 진단	20 초 · 11 초

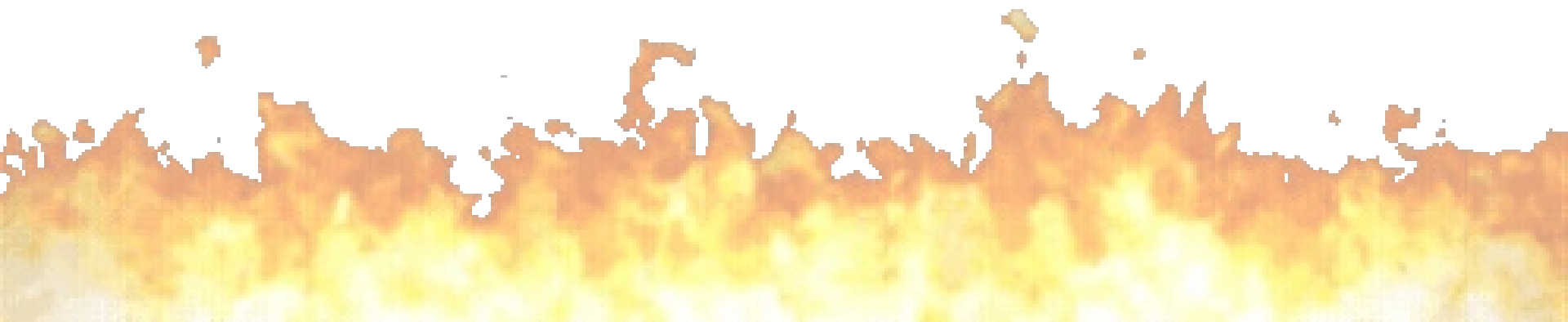
00 번이 시작한 KLUE 준비 (청크 27,943 개 임베딩) 는 12 코어에서 약 15 분 — 1 부 · 2 부 동안 뒤에서 끝난다 .

4 코어 노트북은 2~3 배 걸리므로 00 번을 가장 먼저 실행해 둔다 . 30 번 전에 '✓ KLUE 준비 완료' 확인 .

경로	내용
4 강실습 /docker-compose.yml	db · pgadmin · jupyter · loader (4-1 과 같은 파일에 jupyter 추가)
4 강실습 /jupyter/	JupyterLab 이미지 : numpy · matplotlib · psycopg · pgvector · fastembed · 나눔글꼴
4-2 강실습 /notebooks/labutil.py	DB 연결 · recall 측정 (pg_bench) · 정확한 k-NN · 그래프 한글 설정
4-2 강실습 /notebooks/prepare_klue.py	3 부 데이터 : 4-1 의 load.py · sql/20~22 재사용 + 청크 임베딩
notebooks/00 ~ 13	환경 점검 · 1 부 원리 (NSW · HNSW · IVF · PQ)
notebooks/20 ~ 24	2 부 pgvector (데이터 · HNSW · IVFFlat · 양자화 · 필터)
notebooks/30 ~ 31	3 부 품질 (퓨전 · 분포 진단)

git pull → cd -ai-/4 강실습 → docker compose up -d --build → http://localhost:8888

실습 환경



컴포즈에 Jupyter 추가

1. 서비스 4 개

- db : pgvector(PG17) + 한국어 형태소 분석기 , 5432
- pgadmin : 5050
- jupyter : JupyterLab, 8888 (저장소 루트를 /work 로)
- loader : 4-1 데이터 적재 도구

2. 실행

```
cd -ai-/4강실습
```

```
docker compose up -d --build
```

- 4-1 을 했다면 Jupyter 이미지만 새로 빌드 , db 데이터는 그대로

3.db 에 인덱스 빌드용 설정 추가

- maintenance_work_mem=1GB : HNSW 그래프를 메모리 안에서 빌드
- max_parallel_maintenance_workers=4 · shm_size 1g : 병렬 빌드는 /dev/shm 사용

4.Jupyter 는 로그인 없음

- http://localhost:8888 → 4-2 강실습 /notebooks 가 바로 열림
- 로컬 실습 전용 설정 — 외부에 노출하지 말 것

00 환경 점검과 KLUE 준비

1. 버전 확인

- PostgreSQL 17 · pgvector 0.8.6 · textsearch_ko 1.0
- pgvector 0.8 이상이어야 iterative scan(24 번) 가능

2.KLUE 준비를 백그라운드로 (prepare_klue.py)

- 텍스트 → BM25 역색인 → 지문 임베딩 → 청크 임베딩
- 4-1 을 마쳤다면 앞의 세 단계는 건너뛰고 청크만 (12 코어 약 15 분)

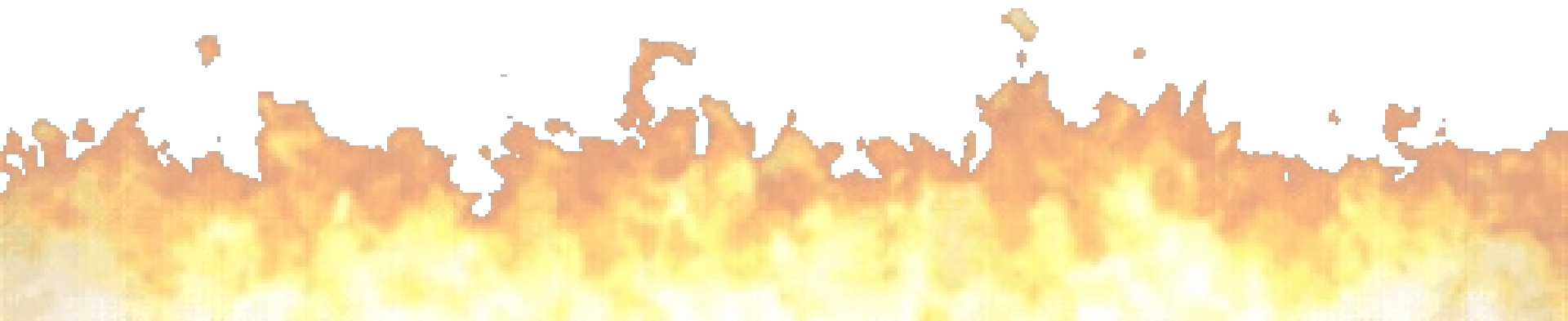
3. 청크 : 문장 경계로 250 자 , 제목을 앞에 붙여 임베딩

- 지문 5,309 개 → 청크 27,943 개 (평균 5.3 개)
- MiniLM 입력 한계 128 토큰 → 지문 통째로는 앞부분만 담긴다 (4-1 결론)

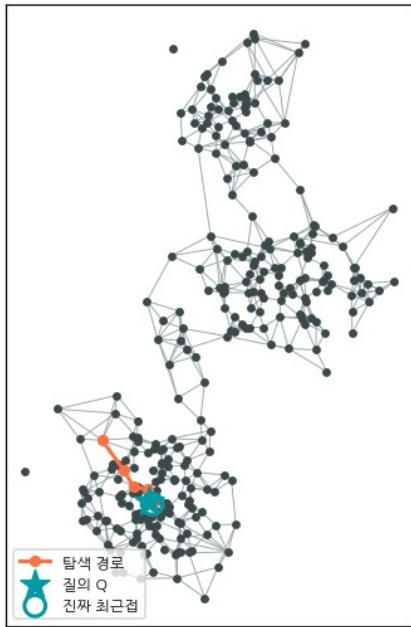
4.labutil.connect() 의 prepare_threshold = None

- psycopg 는 같은 SQL 을 여러 번 실행하면 서버에 prepare 해 두고 계획을 재사용
- SET hnsw.ef_search 를 바꿔도 옛 계획이 쓰여 측정이 틀어진다 → 끄다

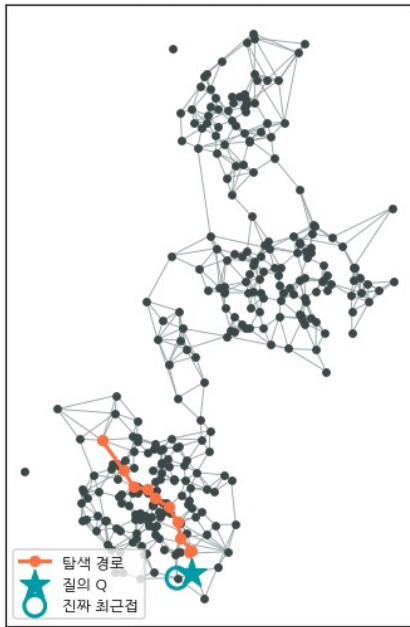
인덱스 원리 — numpy 로 직접



성공



실패: 지역 최적



1. 증분 구축

- 이미 있는 점 중 가장 가까운 M 개와 양방향 , 넘치면 가장 먼 친구를 끊는다
- 먼저 들어온 점일수록 긴 간선 (지름길): 평균 0.57 vs 0.38

2. 그리디만으로는 최근접을 35% 만 맞힌다 (M=4, 점 300 개)

- 오른쪽 : 친구들이 모두 멀어 멈췄는데 그 너머에 정답 = 지역 최적

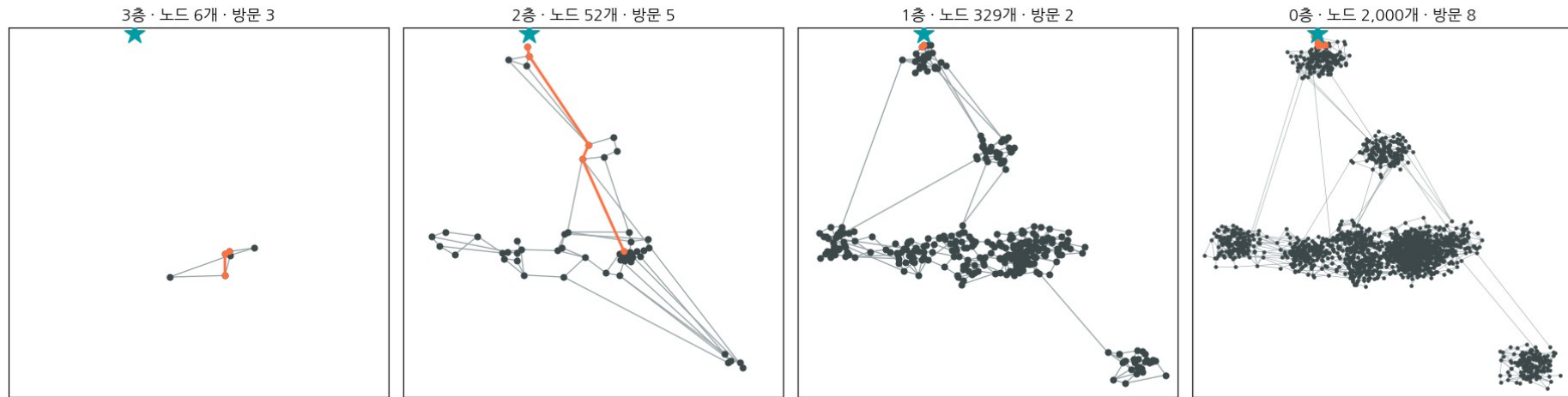
M	진입점에서 도달 가능	ef=1	ef=4	ef=16
2	49%	0.06 / 9 회	0.07 / 11 회	0.43 / 39 회
4	99%	0.35 / 20 회	0.78 / 43 회	0.99 / 64 회
6	100%	0.93 / 46 회	1.00 / 53 회	1.00 / 66 회

칸 = recall@1 / 질의당 거리 계산 횟수 (전수 비교는 300 회). ef 를 키우면 recall 과 비용이 함께 오른다.

M=2 는 군집끼리 이어지지 않아 섬이 생긴다 — 도달할 수 없는 절반은 ef 를 키워도 못 찾는다.

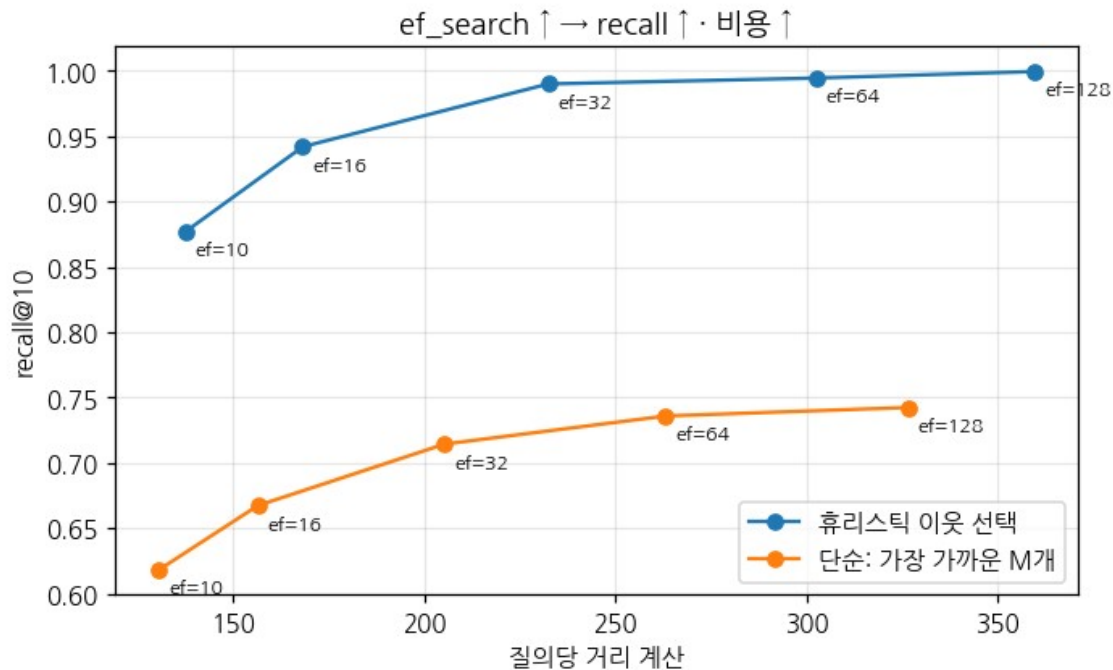
실제 HNSW 의 가지치기는 '가장 먼 친구'가 아니라 다양성 휴리스틱 → 다음 장

HNSW - 층을 내려오며 좁힌다



1. 층 배정 = $\text{floor}(-\ln U \times mL)$, $mL = 1/\ln M$
- 100 만 개, $M=16$: 1 층 62,397 (기대 62,500) · 2 층 3,937 · 3 층 260 · 4 층 15
2. 위층에서 그리디로 멀리 건너가고, 0 층에서 ef 후보로 정밀하게
- 주황 = 방문한 노드. 각 층은 위층이 찾아 준 출발점에서 시작

HNSW - 계층과 이웃 선택의 효과



1. 16,000 개 · 16 차원 : 위층부터 recall 0.99, 0 층부터 (평면 NSW) 0.81

- N 이 16 배가 될 때 질의당 비교는 142 → 232 회 (전수는 16 배)

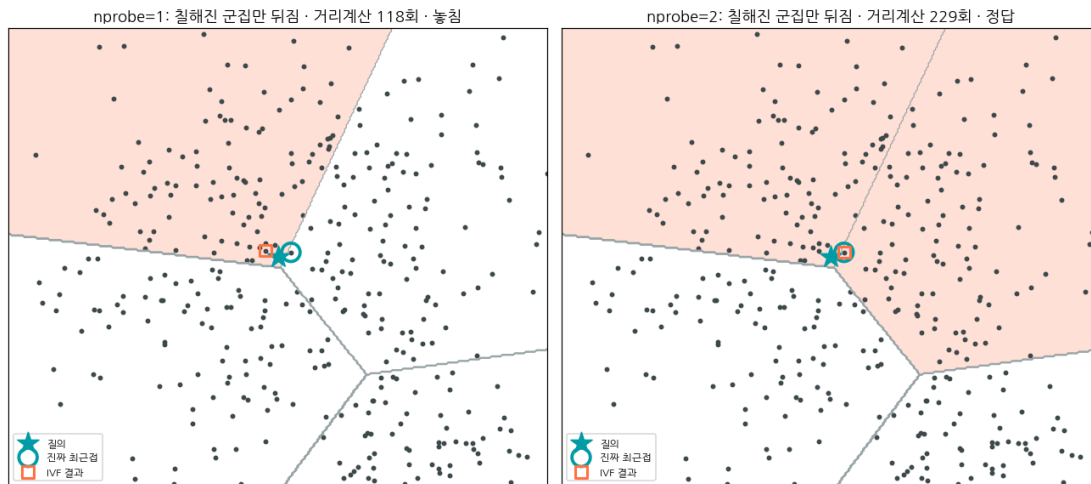
2. 이웃 선택 휴리스틱이 결정적

- 다양성 휴리스틱 0.99 vs 가장 가까운 M 개 0.72 (ef=32)

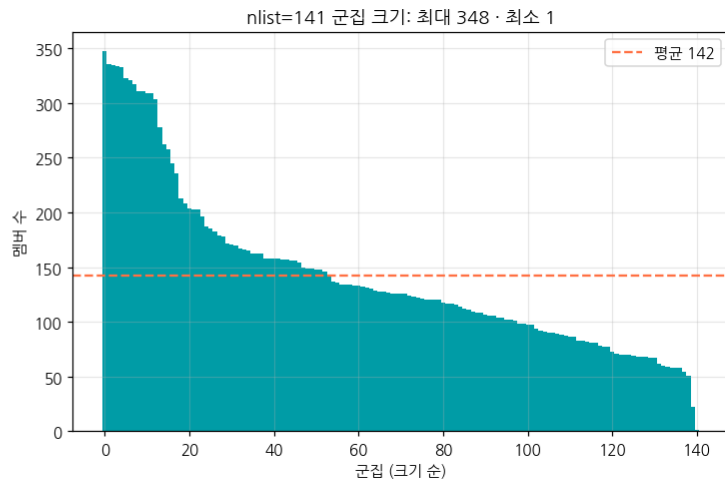
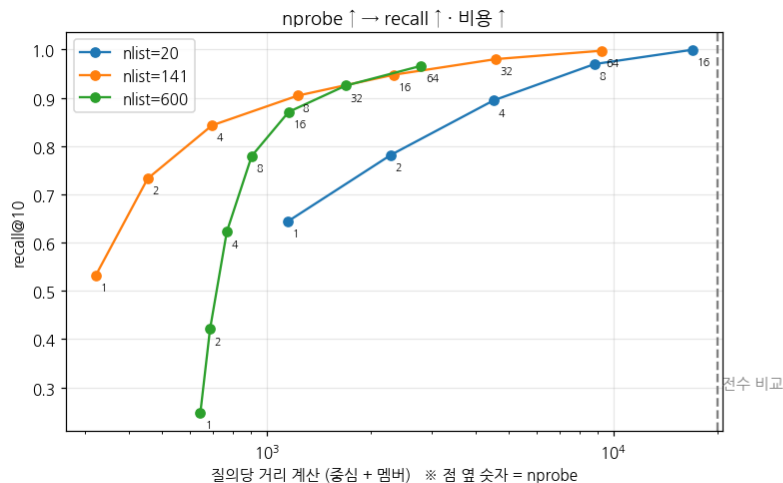
- 가까운 것만 고르면 이웃이 한 군집에 몰려 다리가 약해진다

3. ef ↑ → recall ↑ · 비용 ↑

- ef 10: 0.88 / 137 회 → ef 128: 1.00 / 360 회



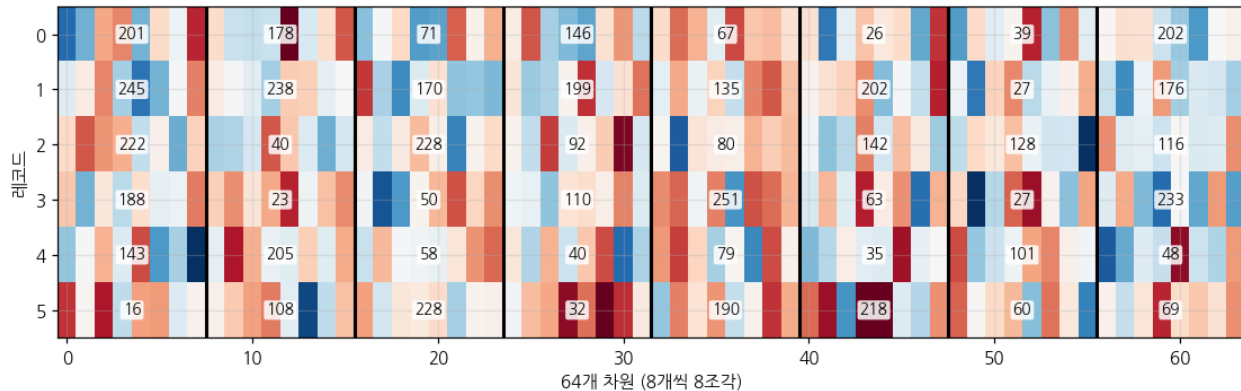
1. 질의는 A 영역, 진짜 최근점은 경계 너머 B 군집
-nprobe=1 은 A 만 뒤져서 놓치고, nprobe=2 로 옆 군집까지 넣으면 찾는다
- 2.k-means 초기값 의존
-시드 20 개 : 무작위 시작은 중심 두 개가 한 군집에 몰리는 실패가 생김
-k-means++ 는 먼 점을 중심으로 골라 실패가 드물다



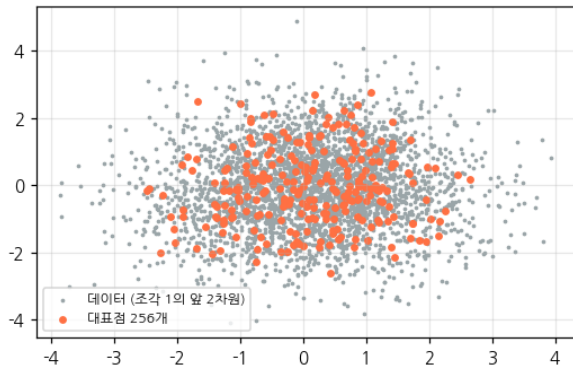
- 32 차원 2 만 개 : $nlist \approx \sqrt{N(141)}$ 곡선이 같은 비용에서 recall 이 가장 높다
- 너무 작으면 (20) 군집이 커서 비싸고 , 너무 크면 (600) 경계가 많아 recall ↓
- 군집 크기 불균형 : 1 ~ 348 개
- 큰 리스트를 뒤지는 질의만 느려진다 — 슬라이드 ' 검색 성능 들쭉날쭉 '
- 학습 시간 $\propto N \cdot k \cdot d \cdot T$, 10% 표본으로 학습해도 recall 0.84 → 0.83

PQ - 8 비트 코드와 거리표 (LUT)

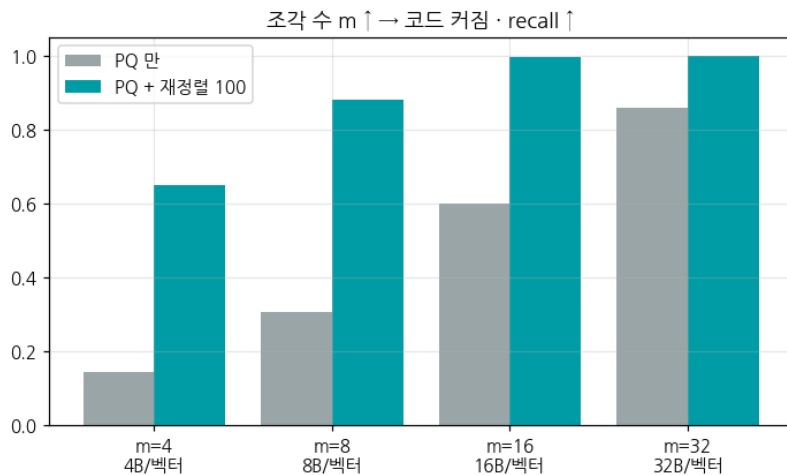
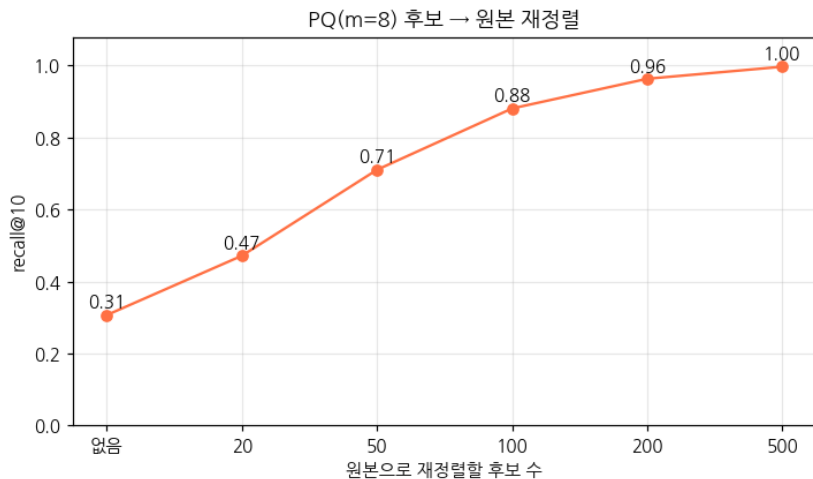
벡터 6개 → 조각마다 가장 가까운 대표점 번호(0~255) = 8바이트



조각 1의 코드북

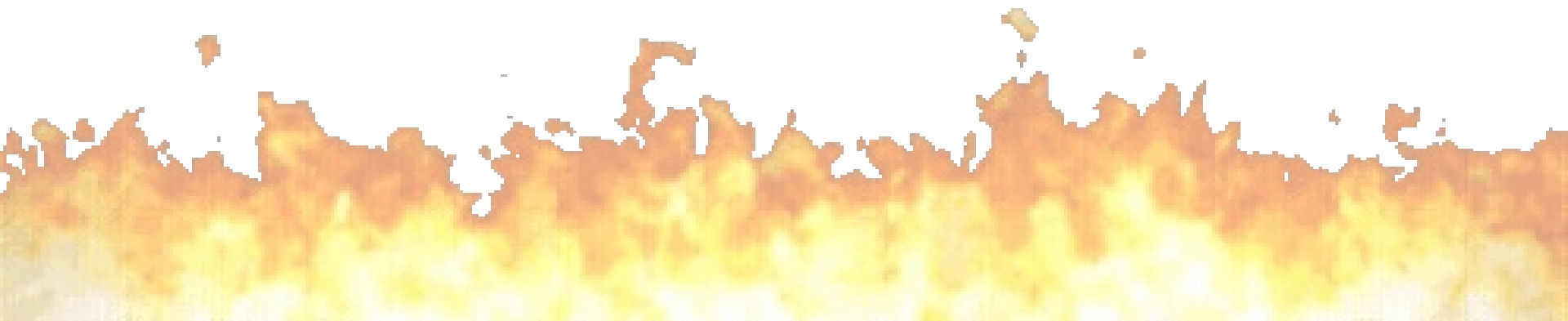


1. 64 차원 → 8 조각 × 256 대표점 → 벡터 1 개 = 8 바이트 (256B 의 1/32)
- '8 차원 압축' 이 아니라 8 바이트 코드 — 차원 축소와 다르다
2. 질의 하나로 8×256 거리표를 만들고, 레코드마다 8 칸을 더한다
- 표 조회 합 144.55 = 복원 벡터와의 거리 144.55 (정확히 같다: 거리²는 조각별 합)
- 원본과의 실제 거리 187.2 — 이 차이가 압축 손실

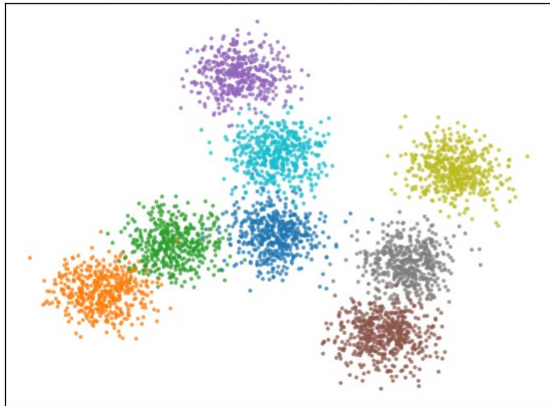


1. PQ 만으로는 recall 0.31 → 후보 100 개 재정렬 0.88 · 500 개 1.00
-m=16(16B) + 재정렬 100 이면 1.00
2. 잔차 (벡터 - 군집 중심) 를 압축하면 같은 8 바이트로 오차 29.6 → 18.8
-IVF-PQ: 군집 고르기 → 잔차 코드로 LUT 비교 → 원본 재정렬

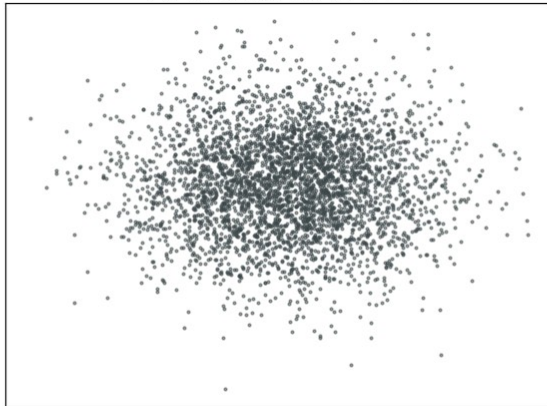
pgvector 인덱스 측정



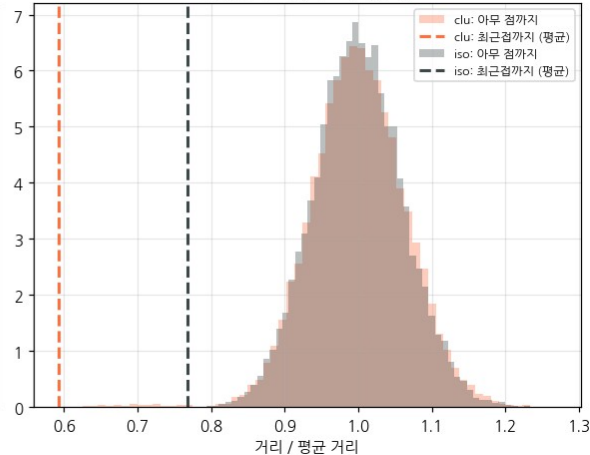
군집형 vec_clu: 군집 8개만 골라 2차원 투영



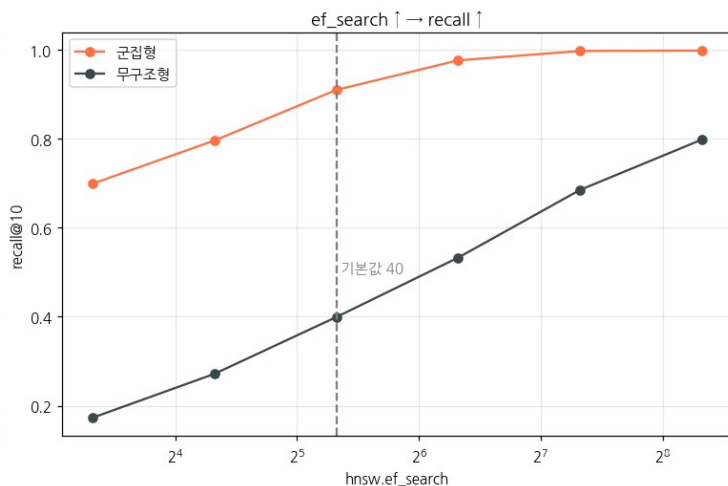
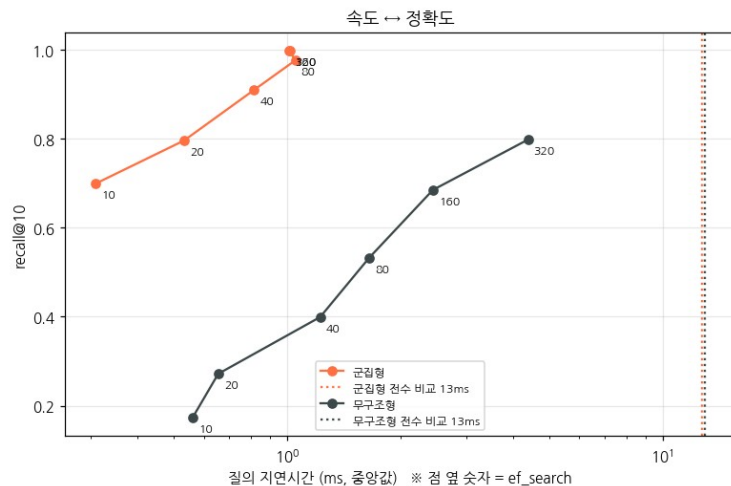
무구조형 vec_iso: 같은 개수를 투영



최근접이 '특별히' 가까운가?

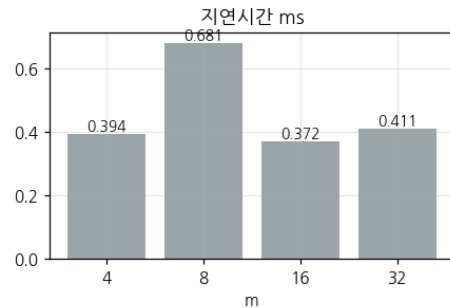
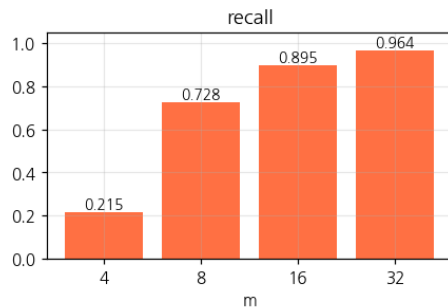
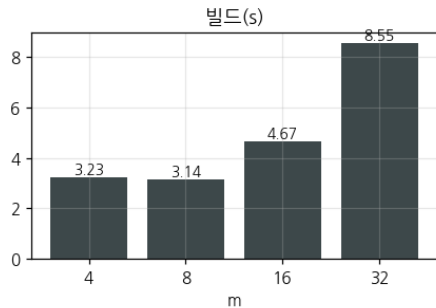
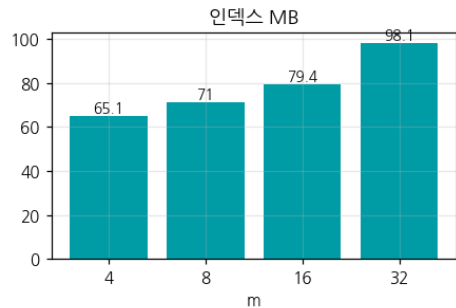


1. 군집형 vec_clu (중심 200 개) · 무구조형 vec_iso (등방 가우시안), 각 10 만 × 128 차원
- 질의 200 개 · numpy 전수 비교로 정답 top-100 → vec_gt
- PG 전수 정렬 (Seq Scan, 약 13ms) 과 정답지 일치 확인
2. 최근접 / 평균 거리 : 군집형 0.59 · 무구조형 0.77
- 고차원에서는 모든 점이 비슷하게 멀다 (거리 집중) → 인덱스의 단서가 약해진다



1. 빌드 5 초 · 인덱스 79MB > 테이블 56MB (벡터 복사본 + 이웃 목록)
2. 기본 ef_search=40: 균집형 recall 0.91 · 무구조형 0.40
- 같은 설정이라도 데이터에 따라 크게 다르다 → 내 데이터로 재고 정한다
3. vector_l2_ops 인덱스는 <-> 에만 쓰인다
- <=> (코사인) 로 물으면 Seq Scan — 인덱스는 빌드한 척도 하나에 묶인다

m ↑ → 이웃 목록이 커져 인덱스·빌드 ↑, recall ↑ (ef_search=40)



1.m 4 · 8 · 16 · 32 → recall 0.28 · 0.72 · 0.92 · 0.97, 인덱스 65 → 98MB

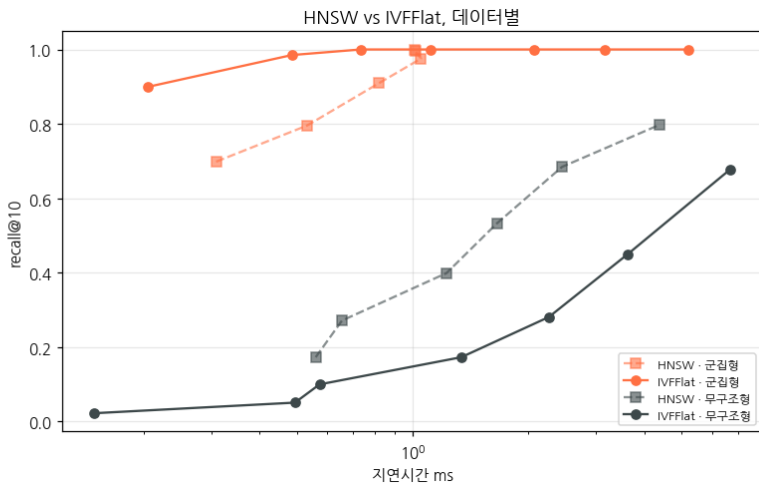
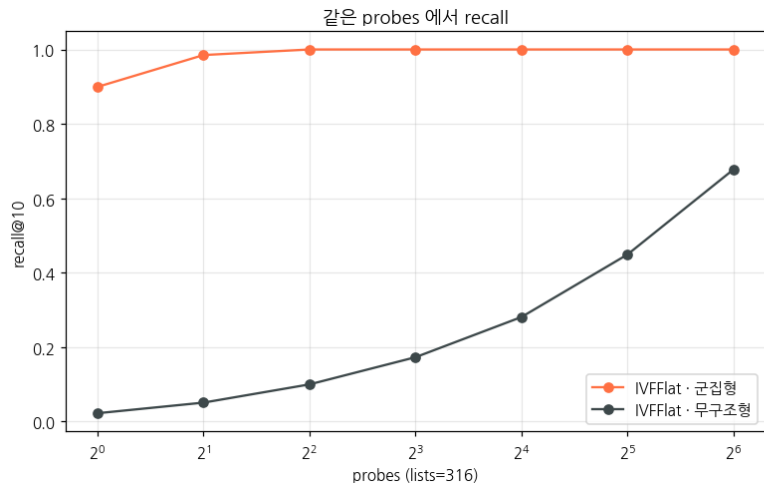
-m 이 너무 작으면 그래프가 약하다 (10 번의 섬)

2.LIMIT 100 인데 ef_search=40 이면 40 행만 온다 — 오류도 경고도 없음

- 재정렬용 후보를 넉넉히 받으려면 ef_search ≥ LIMIT

3.INSERT 즉시 검색된다 , 삭제는 표시 후 VACUUM 이 수선

IVFFlat vs HNSW - 승자는 데이터가 정한다

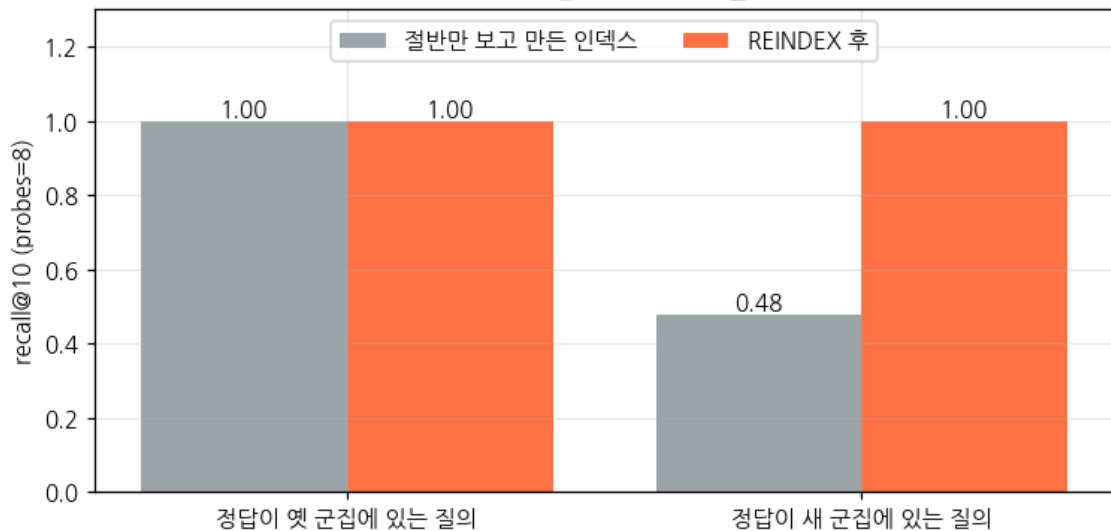


1. probes=1(기본값) 군집형 recall 0.86 — 기본값 그대로 쓰지 말 것
2. 군집형 : IVFFlat(lists=316, probes=4) 이 recall 1.0 으로 HNSW 기본값보다 낫다
- k-means 중심이 진짜 군집과 일치 — 슬라이드 '범주가 뚜렷하면 IVF'
3. 무구조형 : probes=64 로 316 개 중 64 개를 뒤져도 0.67
- 군집이 없으면 '가까운 군집' ≠ '가까운 이웃' — HNSW 가 낫다

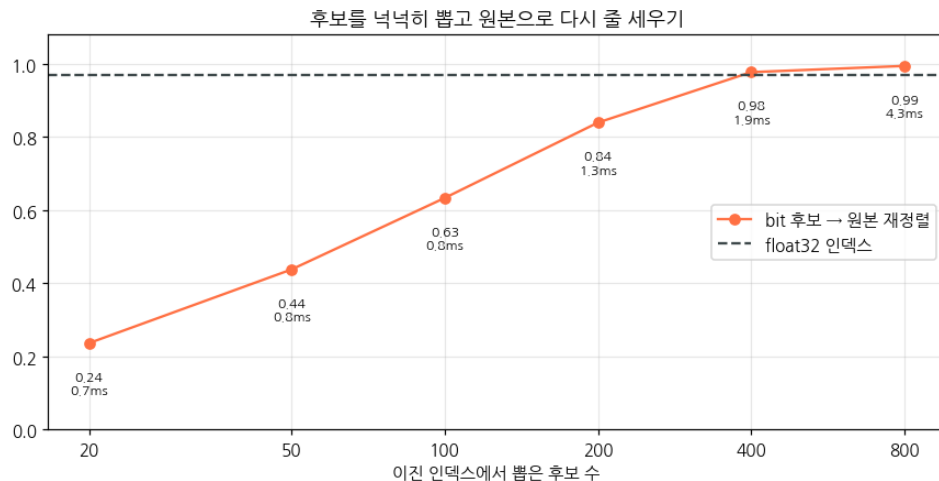
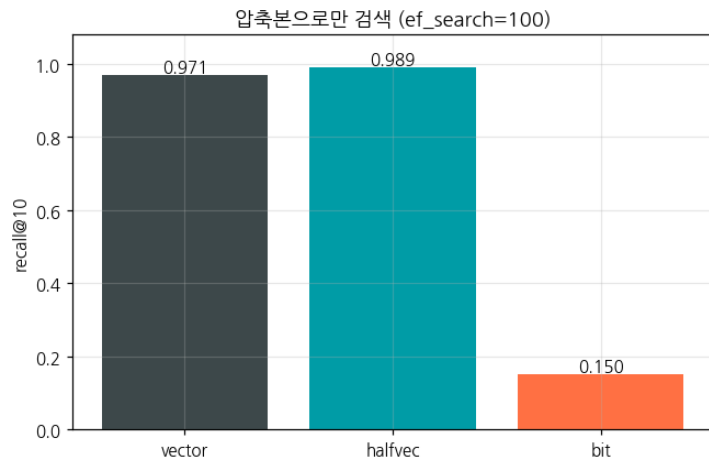
IVFFlat - 분포가 바뀌면 중심이 낡는다



분포가 바뀌면 중심 재학습이 필요하다



1. 군집 100 개일 때 인덱스를 만들고 나머지 100 개 군집의 데이터를 INSERT
- 새 벡터는 옛 중심 중 가까운 리스트에 들어갈 뿐, 중심은 다시 학습하지 않는다
2. 새 군집에 정답이 있는 질의 recall 0.49 → REINDEX(0.8 초) 후 1.00
- 빈 테이블에 IVFFlat 을 먼저 만들고 데이터를 넣지 말 것
3. 빌드 : lists 100 · 316 → 0.5 · 0.8 초 , 1265 → 8 초 (k-means 비용 $N \cdot k \cdot d \cdot T$)



1.pgvector 에는 PQ 가 없다 → 표현식 인덱스로 스칼라 양자화

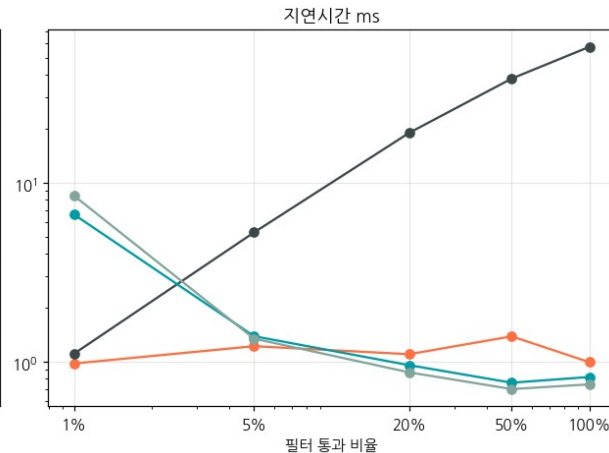
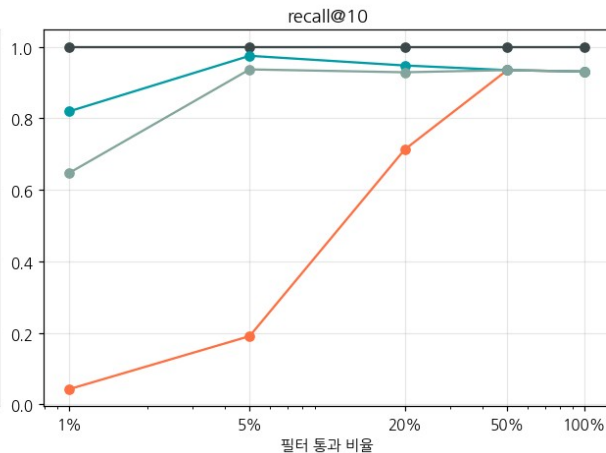
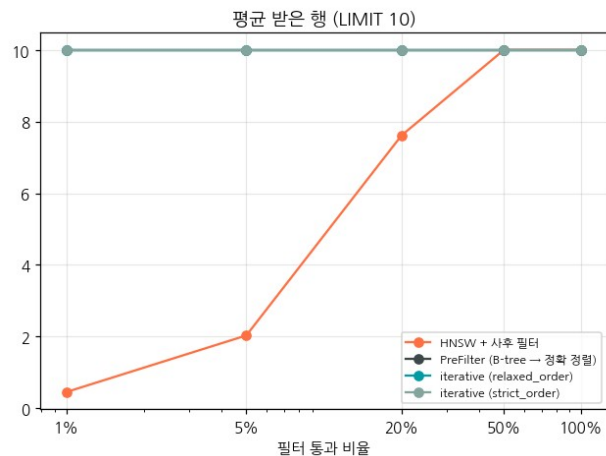
```
USING hnsw ((emb::halfvec(128)) halfvec_l2_ops)
```

```
USING hnsw ((binary_quantize(emb)::bit(128)) bit_hamming_ops)
```

2.halfvec 54MB(-31%) recall 0.981 ≈ float32 0.985 — 가장 손쉬운 절감

3. 1 비트 30MB 는 단독 0.15 → 후보 400 개를 원본으로 재정렬하면 0.975 (2ms)

- 인덱스는 작게 (RAM), 정확도는 원본으로 — IVF-PQ 의 재정렬과 같은 원리



1. 1% 필터 + HNSW: LIMIT 10 에 평균 0.4 행 (PostFilter 함정)

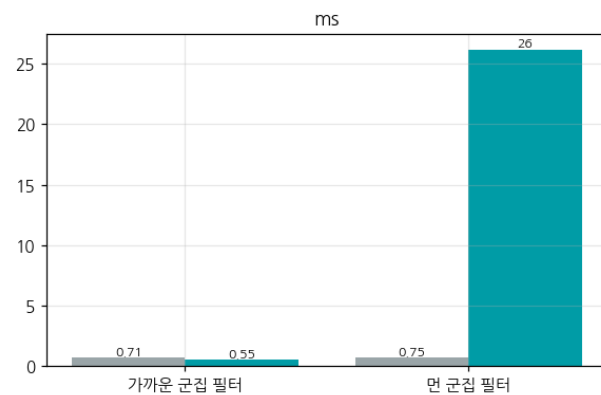
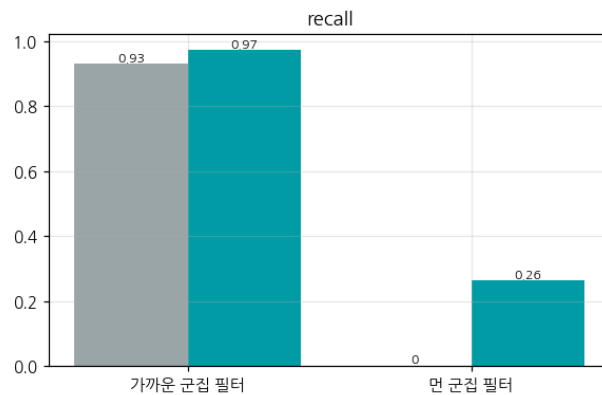
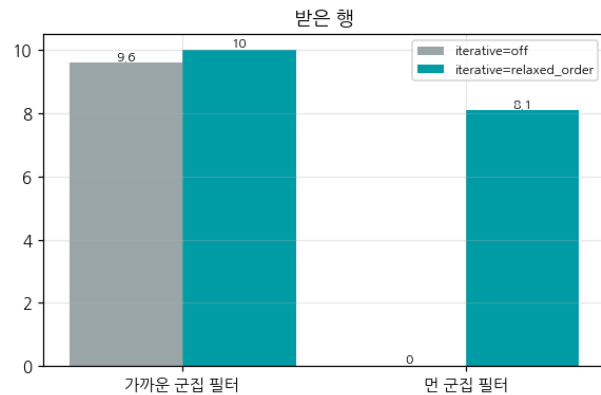
-ef_search 40 개 후보 중 필터를 통과한 것만 남는다

2.B-tree 가 있어도 플래너는 HNSW 를 고른다 → PreFilter 는 직접

WITH c AS MATERIALIZED (SELECT ... WHERE tag < 1) SELECT ... ORDER BY emb <-> q

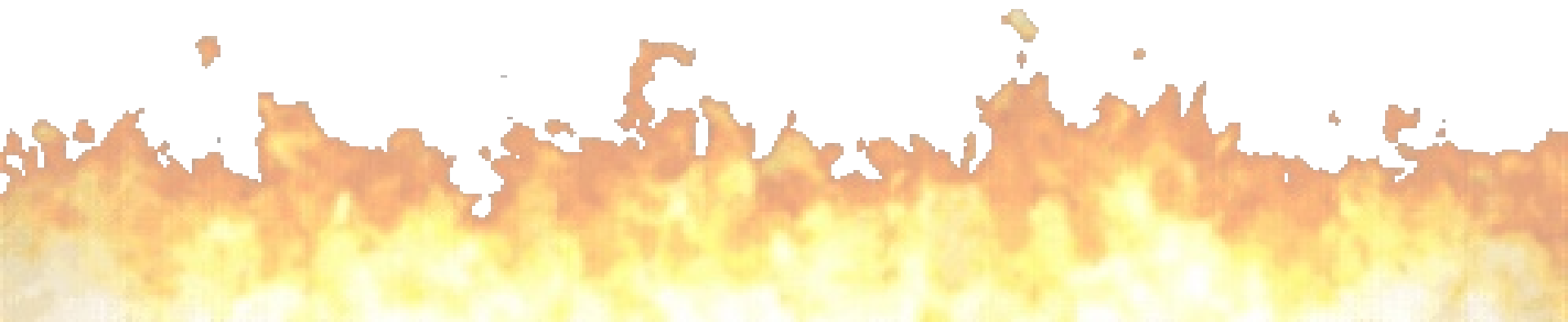
3.SET hnsw.iterative_scan = relaxed_order : 1% 필터도 10 행

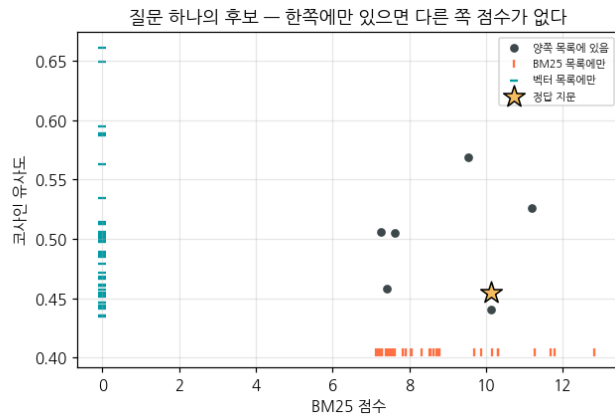
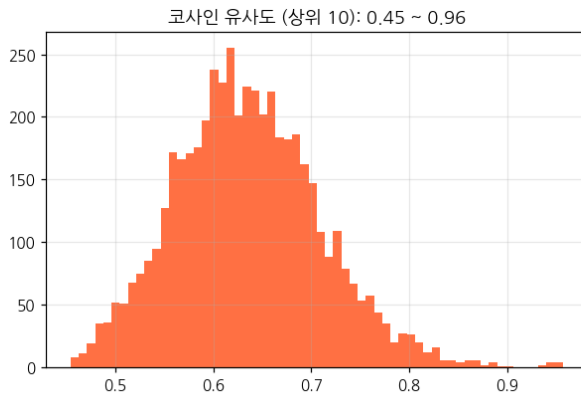
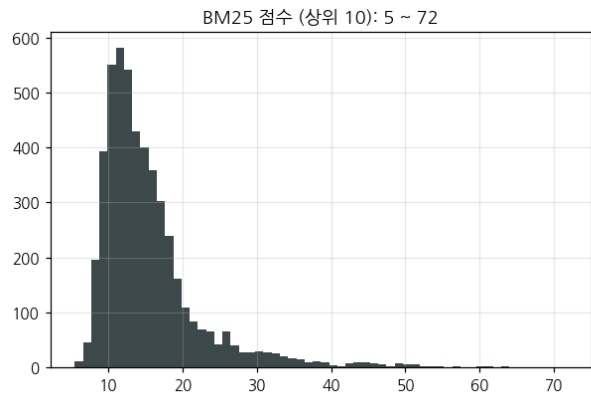
- 지연시간이 선택도에 따라 교차 — 좁으면 PreFilter, 중간은 iterative, 넓으면 사후 필터



1. 선택도가 같아도 (0.5%) 필터가 질의와 같은 방향이면 쉽다
 - 가까운 군집 필터 : 사후 필터로도 9 행 · 0.8ms
 - 먼 군집 필터 : iterative 로도 recall 0.27 · 26ms (탐색 상한 2 만 행까지 헤맨다)
2. 부분 인덱스 : CREATE INDEX ... USING hnsw (emb) WHERE tag = 7
 - recall 1.0 · 0.3ms — 4-1 의 '부분집합 그래프는 불가능' 은 PG 에서는 가능
 - 값이 적고 고정된 필터 (테넌트 · 카테고리) 에 쓴다

검색 품질 — 퓨전 · 분포 진단





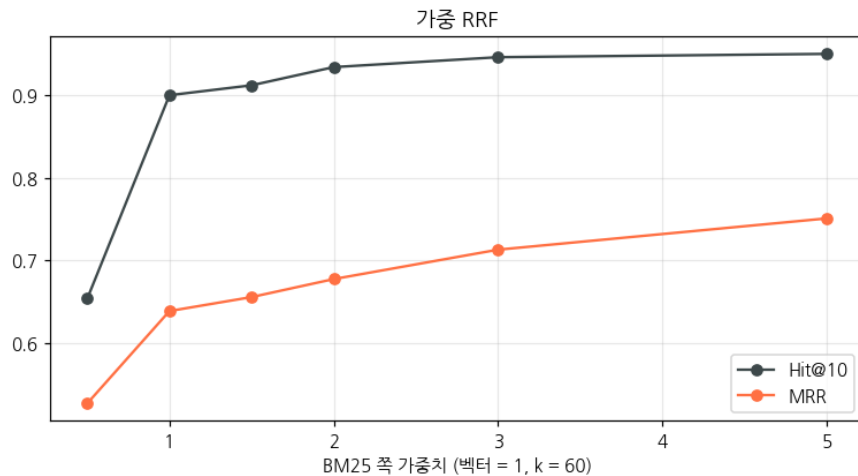
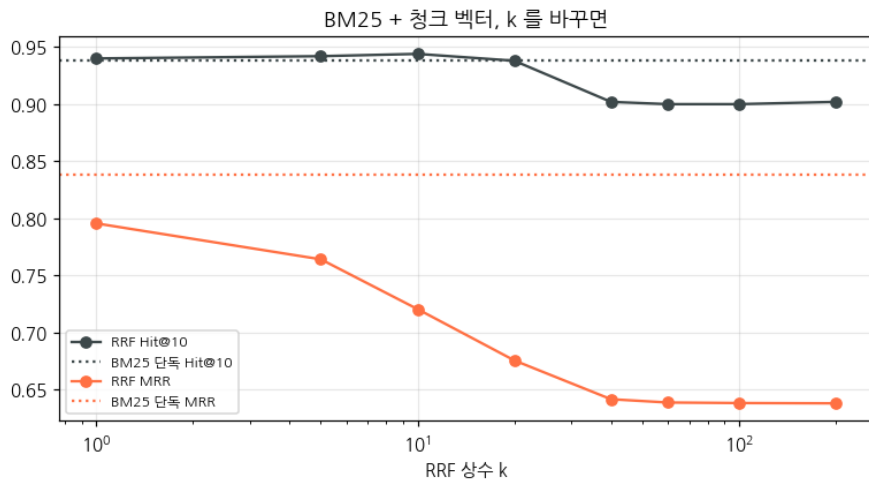
1. KLUE-MRC 질문 500 개 (4-1 과 같은 표본), 각 방법 상위 50 개

- BM25 0.938 / 0.838 · 벡터 (지문 통째) 0.386 / 0.208 · 벡터 (250 자 청크) 0.500 / 0.317 (Hit@10 / MRR)

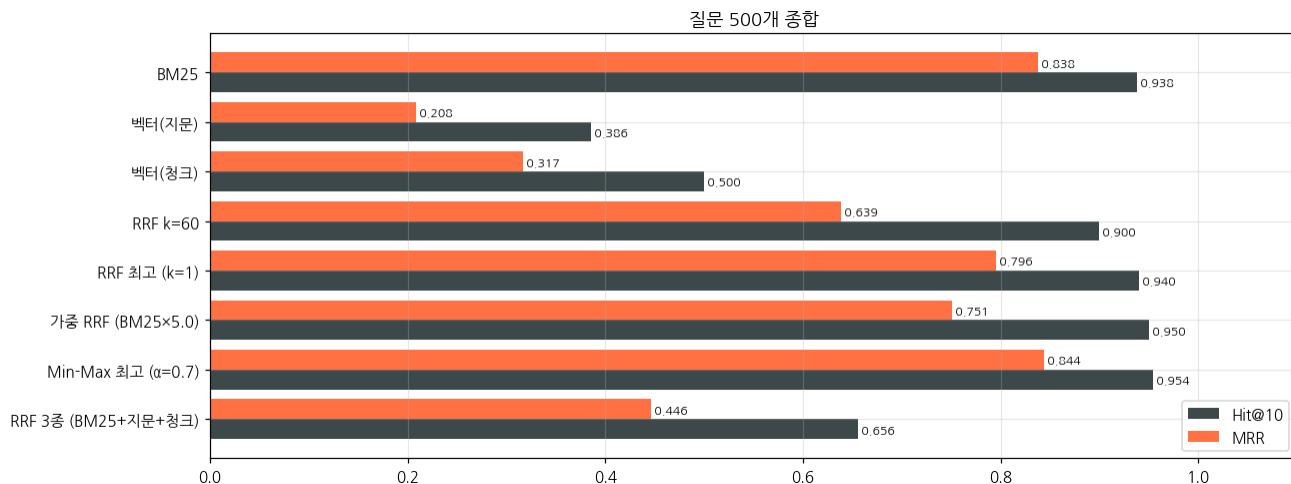
2. BM25 5~72, 코사인 0.45~0.96 → 그냥 더하면 BM25 가 다 정한다

- 한쪽 목록에만 있는 문서는 다른 쪽 점수가 아예 없다

RRF - 순위만 쓴다



1. $1/(k+rank)$ 는 쌍곡선 : 변곡점이 없고 k 는 앞 순위 쓸림 정도만 정한다
 - $k=0$ 이면 10 위가 1 위의 $1/10$, $k=60$ 이면 0.87
2. 흔한 $k=60$ 은 $MRR\ 0.839 \rightarrow 0.639$
 - 약한 벡터 목록과 동등하게 섞기 때문 (4-1 25 번과 같은 현상)
3. $k=1 \rightarrow 0.940 / 0.796$, BM25 가중치 5 $\rightarrow 0.950 / 0.751$
 - Hit@10 은 회복하지만 MRR 은 BM25 단독을 넘지 못한다



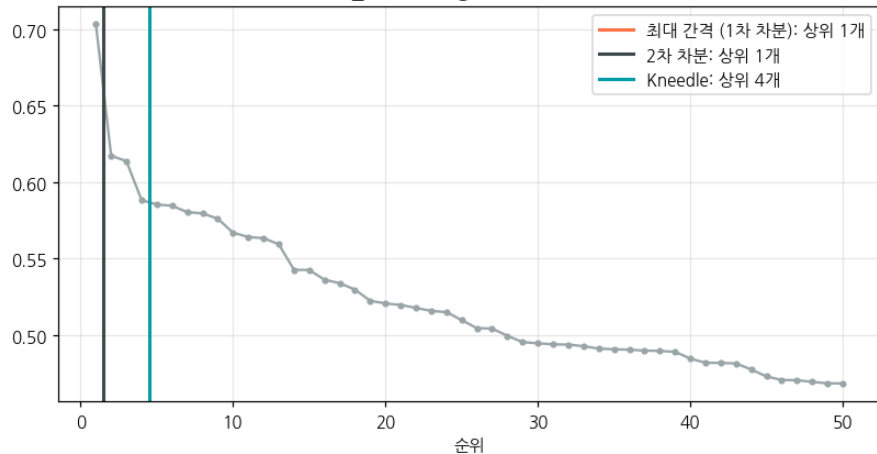
1. Min-Max $\alpha=0.7$: Hit@10 0.954 · MRR 0.844 — BM25 단독을 넘은 유일한 조합

- 점수 크기를 살려서, BM25 가 확신하는 질문은 두고 애매한 질문만 벡터가 바꾼다

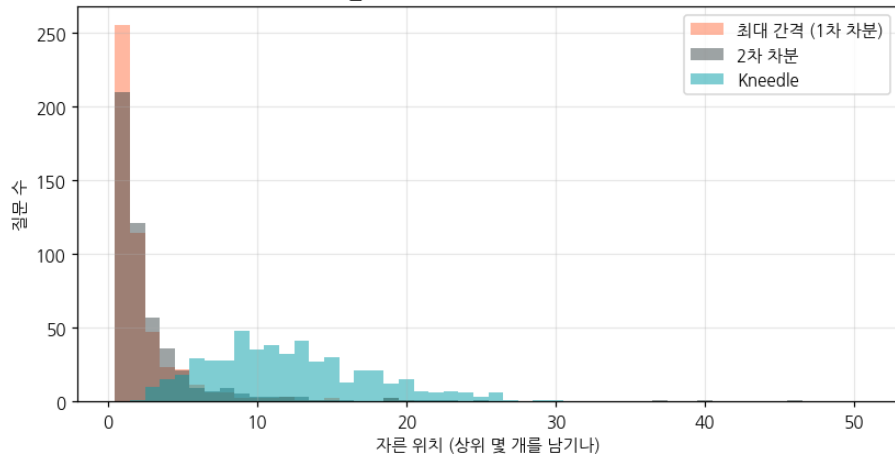
2. 약점 : 1 위가 압도적이면 나머지가 0 근처로 눌린다 · α 는 데이터마다 다시 찾는다

- 평가셋을 나눠 고르고 확인해야 과적합을 피한다

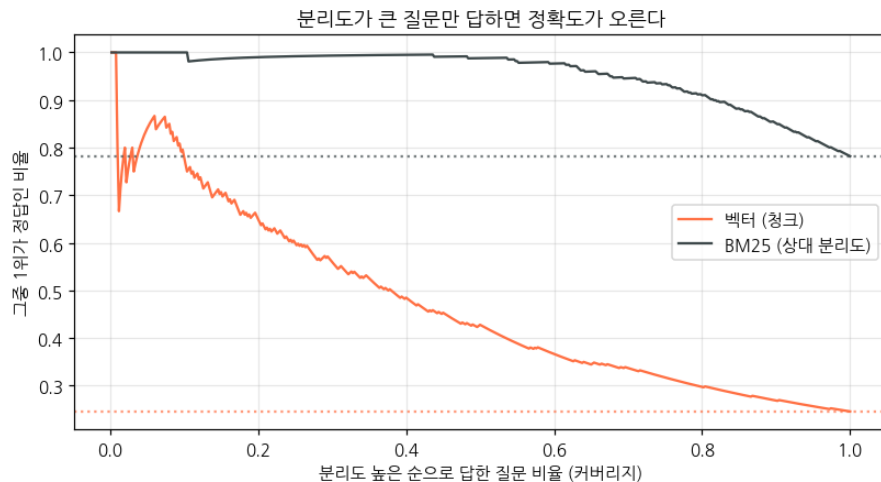
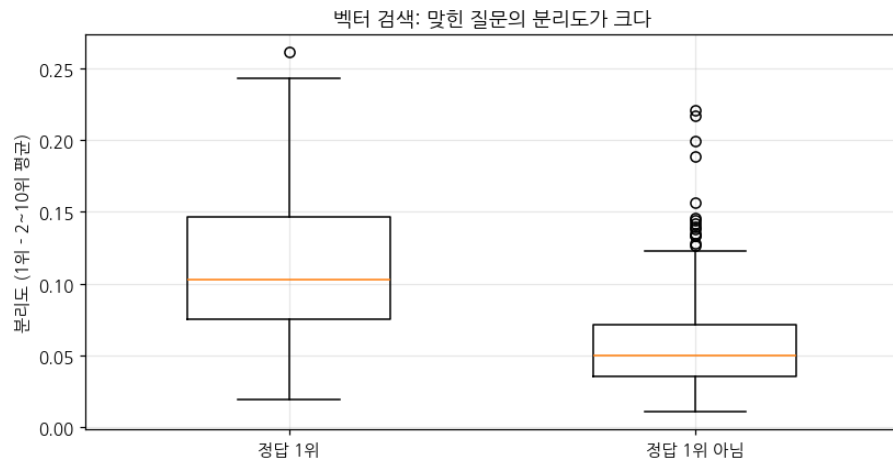
한 질문에서 세 방법이 자르는 위치



질문 500개에서 자른 위치의 분포



1. 맞는 질문은 1 위가 튀어나온 곡선, 틀린 질문은 밋밋한 곡선
2. 무릎점 세 방법 (질문 500 개)
 - 최대 간격 · 2 차 차분 : 대부분 1~2 위 뒤에서 자름 — 2 차 차분은 가끔 중간 요철에 반응
 - Kneedle : 평균 12 개를 남기고 정답이 남는 비율 0.52 (간격 0.33)

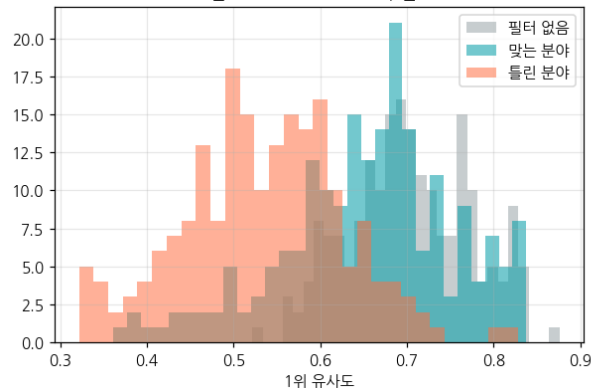


1. 분리도 = 1 위 점수 - 2~10 위 평균
- 맞힌 질문의 분리도가 뚜렷하게 크다
2. 분리도 상위 20% 만 답하면 벡터 1 위 정확도 0.25 → 약 0.7
- BM25 도 같은 경향 (상위 40% 에서 거의 1.0)
- 낮으면 '모르겠다' · BM25 로 넘기기 · 질의 재작성 — 정답셋 없이 운영 중에 켤 수 있다

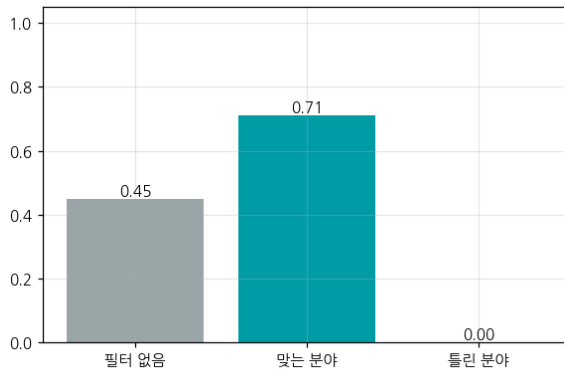
필터 적합성 - 틀린 필터는 유사도로 드러난다



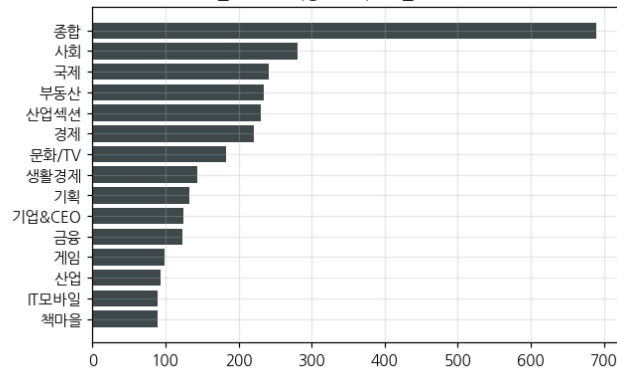
틀린 필터 → 1위 유사도가 확 떨어진다



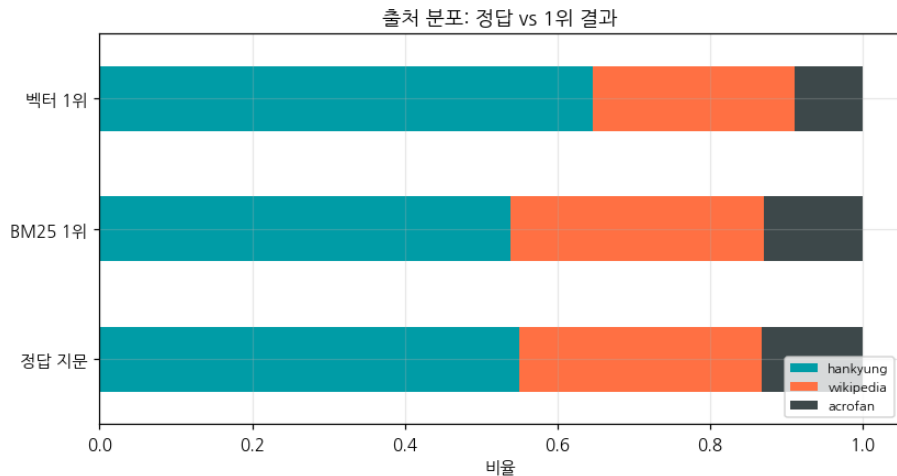
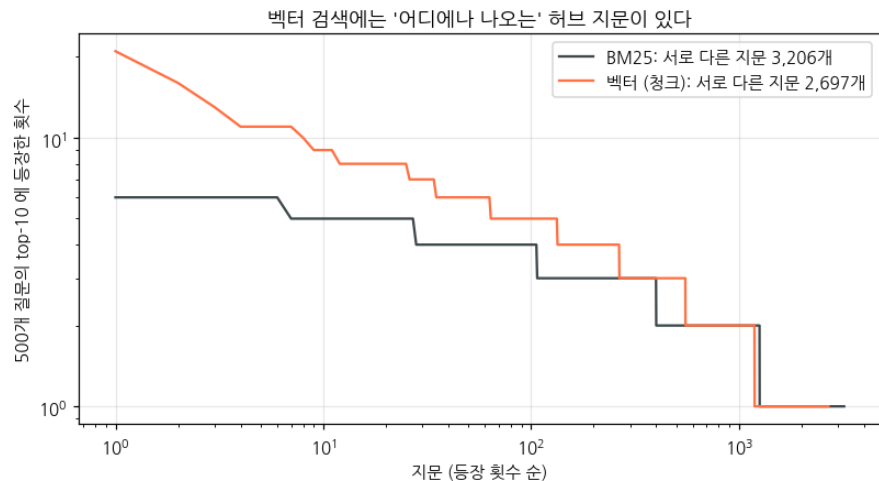
Hit@10



분야별 지문 수 (상위 15) — 필터 후 남은 후보

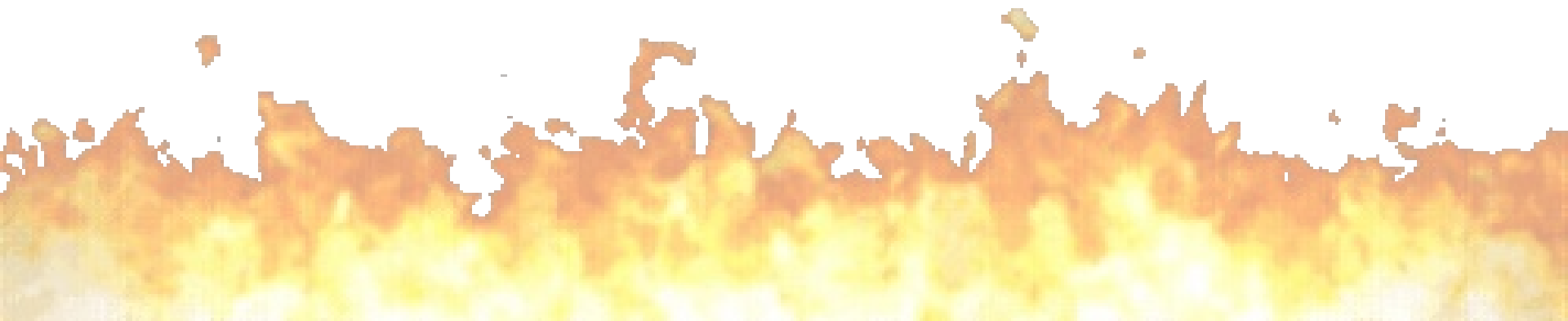


1. 뉴스 질문 200 개 : 필터 없음 · 정답 분야 필터 · 엉뚱한 분야 필터
- 맞는 필터 Hit@10 0.45 → 0.71 (후보가 수백 개로 줄어든다)
- 틀린 필터는 1 위 유사도부터 급락 — 슬라이드 '유사도가 낮으면 시맨틱 불일치'
2. 유사도 절대값은 믿을 수 없다
- 관련 없는 질문-체크 쌍도 코사인 평균 0.15 (이방성) — 고정 임계값보다 곡선 모양



1. 벡터 검색 : 지문 하나가 질문 21 개의 top-10 에 등장 (BM25 는 최대 6)
 - 질문과 상관없이 ' 평균적으로 비슷한 ' 허브가 다양성을 떨어뜨리고 정답을 밀어낸다
2. 출처 분포 : 벡터 1 위의 한국경제 비율 65% vs 정답 55%
 - 결과가 한쪽 출처로 쏠리면 코퍼스 · 청킹 구성을 점검 — 슬라이드 ' 코퍼스 커버리지 '

정리



슬라이드	실습 결과
같은 속도에서 recall 은 HNSW 가 최고	데이터에 따라 다르다 . 군집이 뚜렷하면 IVFFlat 이 낫다 (22 번)
범주가 뚜렷하면 IVF	맞다 . 범주가 없으면 IVF 는 특히 크게 무너진다 (0.67 @ probes 64)
이웃 = 가장 가까운 M 개	실제는 다양성 휴리스틱 : 0.99 vs 0.72 (11 번)
HNSW 는 metric-agnostic	만든 인덱스는 척도 하나에 묶인다 : $\leq$ 는 L2 인덱스 못 탐 (21 번)
512 차원을 8 차원으로 압축	8 차원 벡터가 아니라 8 바이트 코드 (13 번)
RRF 의 k 로 변곡점 조절	$1/(k+rank)$ 에는 변곡점이 없다 . k 는 쏠림 정도 (30 번)
2 차 미분으로 변곡점	이산 점수에서는 잡음에 민감 . 간격 · Kneedle 도 함께 (31 번)
부분집합 그래프는 불가능	PG 부분 인덱스로 가능 : recall 1.0 (24 번)

정리와 문제 해결

1. 끝나면

```
docker compose down          # 컨테이너만 정지 ( 데이터 유지 )  
docker compose down -v      # 데이터까지 삭제
```

2. 자주 겪는 문제

- recall 이 모든 설정에서 같다 → 다른 노트북이 남긴 인덱스 . 첫 셀부터 다시
- 30 번 'KLUE 준비 ' 오류 → 00 번 진행 확인 셀 · prepare.log
- 그래프 한글이 □ → docker compose build jupyter

3. 과제

- 21 · 22 번을 합성 벡터 대신 KLUE 청크 임베딩 (384 차원) 으로 반복해 보기
- 31 번의 분리도가 낮은 질문만 BM25 로 넘기는 라우터를 만들고 Hit@10 비교